

Service Contracts with Limited Memory: Participation, Randomization, and Exact Design

Anonymous manuscript for Operations Research

Abstract. Service agreements can reach the same operating state with different accepted obligations, while the execution system retains only a small service code. We solve the resulting memory-design problem under two participation institutions. Approval before a private draw permits lotteries that reduce the loss from a restricted code budget; realization-by-realization approval eliminates that advantage at every budget. For heterogeneous quadratic renewal contracts, the globally optimal lottery codebook has cap-anchored levels except for a continuously optimized highest level. An exact example shows why discretizing that level at the caps fails. We prove that the optimized terminal costs retain a Monge inequality and identify the resulting matrices with the classical easy-direction staircase-search setting. On ordered rational inputs, standard SMAWK then computes every requested budget frontier with linear rational work per layer, including codebook reconstruction. A resource-pricing formulation separates writable bits, read-only representation, evaluation work, and compilation. A bounded-overflow mechanism makes the institutional tradeoff explicit. Exact checks, independent matrix-search substitution, finite-network solver comparisons, and scale and memory measurements assess the implementations. The broader compact-response and exact-memory theory is retained in the electronic companion and preservation archive.

Keywords: service contracts; limited memory; Monge optimization.

Subject classifications: Dynamic programming: service-code design; Programming: exact resource allocation.

Area of review: Optimization.

1 Introduction

A service agreement may be renewed under several customer conditions and then handed to a common execution system. The renewal desk knows the customer’s accepted obligation. The execution system may receive only a short service code. This separation makes history a design resource: retaining fewer codes simplifies deployment, but forces some accepted continuations to share an action. The relevant question is not merely how to represent an unconstrained optimum. It is how much service value each additional code buys, and how that value depends on when participation is checked.

We study that question in a renewal model with heterogeneous continuation caps and a common terminal operating state. The provider commits to deliver an expected total equal to the weighted mean of the branch caps. This saturated promise is substantive: it makes every branch’s accepted continuation bind. Intermediate service is costly, and terminal service has an increasing concave operating return. A writable code is the only branch-specific information retained at the common terminal state. The public state, read-only program, and numerical precision are distinct resources, not additional uncharged history channels.

Two institutions yield different design prescriptions. Under expected participation, the customer approves the continuation before a private service-code draw. Adjacent-code lotteries can improve value even when a deterministic codebook has already been optimized. Under realization-by-realization participation, every draw must respect the same cap. Randomization then provides no improvement at any code budget in this renewal architecture. Thus the value of randomization is an institutional choice, not a universal property of limited-memory control. A concrete advance-service and dispatch-ticket mechanism states who observes the draw, when exit is available, and which constraints a realization may exceed. A bounded-overflow variant connects the two extremes without interpreting an expected cap as a hard billing limit.

Our central optimization result jointly chooses the service levels and their lotteries. For quadratic terminal returns and heterogeneous quadratic intermediate costs, an optimal codebook can anchor every nonhighest level at a branch cap. The highest level remains continuous. We give an exact example in which restricting it to the caps misses the global optimum. This structural reduction yields a finite dynamic program with exact optimization of each continuous terminal interval. The result is a global frontier for every requested memory budget, rather than a fixed-codebook evaluation or a stationary-point calculation.

The computational consequence rests on a second model-specific fact. After optimizing the highest level continuously within an interval, the terminal-cost array still satisfies a Monge inequality. The proof uses a single-crossing identity in which heterogeneous intermediate curvatures cancel; convexity of the individual minimizations alone would not establish the result. The resulting one-sided feasible domains are classical easy-direction staircase matrices. We make that identification explicit and apply standard matrix searching, with complete tie and reachability proofs matched to the implementation. The linear-work guarantee is in exact rational arithmetic on already ordered inputs, not a new general staircase-search algorithm or a claim of uniform wall-time superiority.

The complete budget frontiers become primitives of a resource-allocation decision. A designer can price writable bits, a chosen read-only representation, evaluation work, and amortized compilation. This accounting prevents a small writable alphabet from being mistaken for a small total implementation. The electronic companion retains the broader compact-response compiler, tight response-size bounds, piecewise-quadratic

closure, behavioral minimization, and shared-table comparison under their original assumptions. They explain where exact memory requirements come from; they are not additional headline results competing with the service-design frontier in the present article.

We evaluate the remaining algorithmic claims separately. Exact regression tests check costs, reconstructed controllers, and the all-submatrices completion condition. Substituting an independently authored matrix-search routine isolates implementation dependence. A standard linear-programming solver checks finite reduced network formulations. Larger same-input experiments measure runtime and memory rather than infer practical speed from an asymptotic bound. These experiments are synthetic verification and performance studies, not calibrated evidence about customer demand or willingness to accept a lottery.

2 Relation to Established Optimization Methods

Contracts and retained information. Continuation promises and participation are established state variables in dynamic contracting (Spear and Srivastava, 1987; Thomas and Worrall, 1988). Finite policy graphs also have a contractual history (Zhang, 2012). Our model has public exogenous branches and no private reports; it is not an incentive-compatibility result for hidden types. Its resource constraint is the information crossing a specified execution boundary. Classical behavioral minimization determines exact equivalence once a finite output transducer has been constructed (Givan et al., 2003; Peyrière, 2023). Here the design problem deliberately operates below the exact alphabet size.

The companion maps the unfolded accepted-service problem invertibly to laminar resource allocation. Tree-constrained allocation, polymatroid optimization, and normalized memoization are inherited tools (Mjelde, 1983; Tang, 1990; Federgruen and Groenevelt, 1986; Groenevelt, 1991). The compact-input event and storage bounds concern a repeated-subtree subclass, not a new solution to arbitrary laminar allocation. Those results support the resource accounting but are not needed to solve the renewal frontier developed below.

Four algorithmic layers. Quadrangle-inequality dynamic programming is classical (Yao, 1980). Searching a full totally monotone matrix is the SMAWK result of Aggarwal et al. (1987). Partial matrices require a separate comparison: Klawe and Kleitman (1990) developed almost-linear generalized searching, and Chan (2021) obtained expected linear time for the harder Monge staircase setting. Submatrix-query data structures for full and partial Monge matrices form a further related literature (Kaplan et al., 2017). These problems differ from a single completed row-minimum pass; citing only full-matrix SMAWK would obscure the relevant prior work.

Most importantly, the introduction of Chan (2021, pp. 2–3 of the author version) already separates staircase orientations that admit direct completion and SMAWK from the harder orientations. Our terminal array has feasible suffixes that shrink with increasing row index. Under the row-minimum convention used below, this is the concave-Monge falling-staircase easy case. The program array has feasible prefixes that expand. Reversing both its row and column orders converts it to the same easy case and preserves the Monge inequality. We give direct prefix and suffix proofs so that the actual left-tie convention, rather than an implicit

reversal of tie order, is audited. Neither proof claims a new general completion method.

The distinction also explains the complexity comparison. No generic inverse-Ackermann barrier is overcome here. The branch order fixes a favorable nesting direction; completed row preferences therefore cannot reverse in the direction forbidden by total monotonicity. Other partial domains, the opposite orientation under a fixed inequality convention, and online dependencies within a searched layer are outside our theorem. All predecessor values in a layer are known before its search begins. The quantitative contractual work is to establish the finite-cost inequalities after continuous minimization and a constant-work exact oracle. Once those facts hold, the linear search is a classical consequence.

Emmerich (2026) provides a recent, different application: integral R2 subset selection is reduced to a Monge dynamic program and accelerated through monotone and staircase search. That work reinforces that this generic algorithmic progression is not distinctive to service contracts. It does not supply the present cap-anchoring reduction, institutional comparison, or continuously minimized contractual terminal costs. Table 1 separates these contributions theorem by theorem. We attribute the search machinery positively rather than assign novelty to a change in implementation order.

3 The Contract and Its Execution Boundary

3.1 A saturated renewal agreement

At the renewal date, a public branch $j \in \{1, \dots, k\}$ is observed with probability $\pi_j > 0$, where $\sum_j \pi_j = 1$. Its continuation cap is $0 < b_1 < \dots < b_k < 1$. The provider chooses an intermediate tier $Y_j \in [0, 1]$ and a terminal tier $C_j \in [0, 1]$. All branches subsequently reach the same terminal public vertex. Unit payment coefficients and unit discounting make total remaining payment $Y_j + C_j$. A fixed service-benefit continuation minus the customer's outside continuation gives the cap. At the root the provider has committed to

$$\sum_j \pi_j \mathbb{E}[Y_j + C_j] = \bar{b}, \quad \bar{b} := \sum_j \pi_j b_j. \quad (1)$$

Under expected participation, $\mathbb{E}[Y_j + C_j] \leq b_j$ for every j , with expectation conditional on that observed branch and taken before the internal draw. Positivity of the probabilities and (1) imply equality separately on every branch. Under pathwise participation, the stronger constraint is $Y_j + C_j \leq b_j$ almost surely. The same argument then implies equality almost surely, not merely in expectation.

The operating payoff is $f(C_j) - h_j(Y_j)$, with increasing concave f and convex nondecreasing h_j satisfying $h_j(0) = 0$. General shape assumptions suffice for the deterministic and pathwise results. The global continuous algorithm specializes to the quadratic primitives in (7). The negative intermediate contribution represents the net cost of an advance or expedited tier, while the terminal tier contributes net operating return. It does not mean that the customer has negative service benefit: that benefit has already entered the continuation cap. There are no endogenous public transitions, coupled commitments, or switching costs in the renewal theorem.

3.2 What a writable code means

At the observed branch, an encoder selects a symbol $Z \in \{1, \dots, m\}$. At the terminal vertex the decoder observes only the current public vertex and Z . Its stored program can contain arbitrary read-only model constants but cannot recover the realized branch from those constants. An inherited intermediate tier, a customer identifier with a branch lookup, a retained shared random seed, and a transmitted numerical payment are all additional history channels; none is free in this interface. An m -symbol alphabet requires $\lceil \log_2 m \rceil$ fixed-width writable bits, not m bits. Public-state storage and read-only program storage are counted separately.

Private encoder randomization is allowed under expected participation. Fresh decoder randomness is independent of earlier private choices conditional on the symbol. Any seed that correlates those choices across dates must itself be encoded in the charged symbol. Concavity permits a deterministic terminal decoder at the expected-participation optimum, so a codebook is a collection of terminal levels. The lower bound for exact implementation continues to allow randomized encoders and decoders under this accounting. Distinct full-information terminal actions and strict concavity remain visible assumptions; the exact threshold is not claimed for repeated caps or nonunique full optima.

3.3 An operational mechanism, not an empirical calibration

Consider a stipulated service-batch agreement administered by a renewal desk and executed by a shared dispatch gateway. The desk observes the customer's current outside alternative and approved continuation. It can perform an advance tier and issue one of a small number of dispatch tickets. The gateway implements the terminal tier named by that ticket; its interface does not retain the renewal record. A separate billing audit can retain the contract without making that record an input to the gateway. Limiting tickets can price gateway-state certification, transmission, or a restricted instruction set. This is a documented stylized mechanism, not a claim that a particular industry already uses this architecture.

In the expected-participation version, the offer communicates the advance tier, the terminal ticket levels, and their probabilities. The customer evaluates expected remaining payment before the ticket is drawn and either renews or exits at that review. The protocol commits to the advertised draw. After renewal the desk performs the advance tier, draws the ticket, and sends only that ticket to the gateway. The realized ticket may be disclosed to the customer. "Private" means that the seed is neither a pre-acceptance conditioning variable nor a free future decoder state; it need not mean that realized service is hidden forever. The customer does not receive a new costless exit option after observing an unfavorable draw within this modeled renewal episode.

This version is meaningful only for an ex ante willingness-to-pay or expected-obligation cap and an agreement that permits the stated realization variation. It is not a way to satisfy an ex post enforceable payment ceiling in expectation. A hard billing limit, a service-safety requirement on every draw, or a review after disclosure instead calls for pathwise feasibility, with the deterministic frontier as its optimum. We make no legal assertion about enforceability. The two feasible sets are explicitly different design alternatives.

Repeated customers who may condition a later renewal on past tickets require that history to enter the later public state, cap, or charged memory. The three-date theorem is not an unqualified repeated-game

result. Similarly, a contractible seed disclosed before acceptance changes the conditioning information and hence the participation test. It cannot simply be added as a free source of branch-dependent execution. Section 8 gives a bounded-realization variant for an operator who permits some variation but not the full expected-participation lottery.

4 The Deterministic Service-Code Frontier

Consider three dates. The root tier is fixed at zero. At the intermediate date a branch j is observed with probability $\pi_j > 0$, where $\sum_j \pi_j = 1$. All branches then enter the same terminal public vertex. Payments and discounting equal one. Intermediate and terminal tiers are in $[0, 1]$. Branch j has cap $0 < b_1 < \dots < b_k < 1$, and the root promise is $\bar{b} = \sum_j \pi_j b_j$. At the intermediate date the reward is $-h_j(y)$, where h_j is convex, nondecreasing, and $h_j(0) = 0$. The common terminal reward $f(c)$ is continuous, concave, and strictly increasing on $[0, 1]$. The terminal cap is one. The terminal decoder observes only its public vertex and its memory symbol, not the inherited intermediate tier.

Theorem 4.1 (Heterogeneous renewal frontier). *The unique full-information policy is $y_j = 0$, $c_j = b_j$, with value $J = \sum_j \pi_j f(b_j)$. Exact implementation requires k symbols. With at most m deterministic terminal symbols, an optimal encoding has contiguous nonempty cells in cap order. For a cell $i, \dots, j$, its terminal tier is b_i and its loss is*

$$C(i, j) = \sum_{h=i}^j \pi_h [f(b_h) - f(b_i) + h_h(b_h - b_i)]. \quad (2)$$

The exact minimum loss is $D_m = D(\min(m, k), k)$, where

$$D(s, j) = \min_{s-1 \leq i < j} \{D(s-1, i) + C(i+1, j)\}, \quad D(0, 0) = 0, \quad (3)$$

and all other infeasible entries equal $+\infty$. With a precomputed cell-cost table this requires $O(mk^2)$ arithmetic operations. For quadratic f and quadratic h_j , prefix sums of the probability-weighted coefficients compute each cell cost in constant arithmetic time.

Proof. The root promise is the mean of the branch caps, so every branch cap must bind. Hence $y_j = b_j - c_j$ and $0 \leq c_j \leq b_j$. The branch reward $f(c) - h_j(b_j - c)$ is strictly increasing on that interval. Full information therefore uniquely selects $c_j = b_j$; different terminal actions require different symbols.

Within a memory cell all terminal actions coincide and cannot exceed its smallest cap. Monotonicity selects that cap and subtraction from the full-information reward gives (2). Fix any collection of cell minima. Every branch strictly prefers the largest eligible selected minimum, irrespective of its probability and intermediate cost. Reassigning branches in this way keeps each selected minimum in its own nonempty cell and produces contiguous cells without reducing value. Splitting at the final cell gives (3). Expanding the quadratic terms gives the stated prefix-sum implementation. $\square$

This strengthens and includes the quadratic equal-probability frontier. With $f(c) = 2c - c^2/2$, $h_j(y) =$

$y^2/2$, and $\pi_j = 1/k$, (2) becomes

$$C(i, j) = \frac{2 - b_i}{k} \sum_{h=i}^j (b_h - b_i). \quad (4)$$

For $k = 8$ and $b_j = j/9$, one, two, and three symbols lose $119/162$, $5/18$, and $49/324$, respectively. Two writable bits permit four symbols, not three. The segmentation recurrence is standard; the contract-specific cell loss, ordered-cell sufficiency, and exact memory interpretation are the substantive statements.

The earlier nonsaturated two-branch example also remains useful. With branch caps $1/4$ and 1 , root promise $1/2$, and the same quadratic rewards, the full optimum uses terminal tiers $1/4$ and $3/4$ and has value $27/32$. An optimized public-only table uses common terminal tier $1/4$ and intermediate tiers $0, 1/2$, giving $13/32$. The adaptation gain is $7/16$ relative to the optimized restriction. At the full optimum both inherited intermediate tiers are zero, so making that tier observable does not evade this example's exact-memory lower bound. By contrast, the entire restricted-memory frontier above depends on not allowing a branch-specific intermediate tier to serve as an uncharged code.

5 When Randomization Improves a Restricted Memory Budget

The exact-optimum lower bound in Theorem EC.4.1 does not imply that randomization is useless below that alphabet size. The renewal family permits an explicit answer under the declared conditional-expectation participation institution. At an intermediate public branch, the controller may randomize its terminal symbol; it carries no uncharged seed or branch record. Participation is evaluated before that private draw. Terminal actions may depend only on the terminal public vertex and the retained symbol.

For terminal levels $c_1 < \dots < c_s$, let F_c be the linear interpolant through $(c_i, f(c_i))$. Since f is concave and strictly increasing, this interpolant is concave and strictly increasing on $[c_1, c_s]$; for $s = 1$ it is defined at its sole point. Retain the caps, probabilities, and convex intermediate costs of Theorem 4.1.

Theorem 5.1 (Optimal lotteries for a fixed terminal alphabet). *For a fixed set of terminal levels, a feasible randomized controller exists if and only if $c_1 \leq b_1$. Its optimum at branch j has deterministic intermediate tier and terminal mean*

$$\mu_j = \min\{b_j, c_s\}, \quad y_j = b_j - \mu_j.$$

It randomizes over at most the two adjacent terminal levels bracketing μ_j . The optimal loss with at most m writable terminal symbols is consequently

$$D_m^{\text{ran}} = \min_{\substack{1 \leq s \leq m, \\ 0 \leq c_1 < \dots < c_s \leq 1 \\ c_1 \leq b_1}} \sum_j \pi_j \{f(b_j) - F_c(\min\{b_j, c_s\}) + h_j((b_j - c_s)_+)\}. \quad (5)$$

The minimum allows coincident levels by deleting unused duplicates. It satisfies $D_m^{\text{ran}} \leq D_m$. Equality with the full-information optimum still requires k symbols when the full-information actions are distinct and rewards are strictly concave. Formula (5) optimizes the encoding for each continuous codebook; it is not a claim that the joint codebook problem is solved by the deterministic segmentation recurrence.

Proof. The root promise equals the weighted mean of the branch caps. Every branch cap must therefore bind

in conditional expectation, so $\mathbb{E}(Y_j + C_j) = b_j$. Replace a randomized terminal decoder, conditional on a symbol, by its mean. It uses the same number of symbols, preserves payments, and weakly increases reward by concavity. Replace a randomized intermediate tier by its branch mean; convexity of h_j weakly lowers its cost without altering the encoder's terminal-symbol probabilities. These replacements use exogenous transitions and the absence of switching costs.

For a terminal mean μ in the convex hull of the levels, the largest expected terminal reward is $F_c(\mu)$. The upper hull of the concave points $(c_i, f(c_i))$ consists of adjacent chords; a lottery over their endpoints attains the hull. The branch objective is $F_c(\mu) - h_j(b_j - \mu)$ on $[c_1, \min(b_j, c_s)]$. It is strictly increasing, so its maximum occurs at μ_j . Feasibility requires $c_1 \leq b_j$ for every branch, equivalently $c_1 \leq b_1$. Subtraction from full information gives the formula. Compactness of symbol probabilities and levels gives attainment, with redundant symbols deleted. Deterministic encodings are admissible randomized encodings, yielding the inequality. The exact-optimum statement follows from the strict-concavity argument in Theorem EC.4.1, not from the restricted-alphabet formula alone. $\square$

Proposition 5.2 (A strict, exactly solved two-symbol separation). *In the quadratic renewal model, take three equiprobable branches with caps $1/4, 1/2, 3/4$, $f(c) = 2c - c^2/2$, and $h_j(y) = y^2/2$. Then*

$$D_2 = \frac{1}{8}, \quad D_2^{\text{ran}} = \frac{1}{96}, \quad D_3 = D_3^{\text{ran}} = 0. \quad (6)$$

Two symbols thus reduce the randomized loss to one twelfth of the best deterministic loss, without reducing the three-symbol requirement for exact implementation.

Proof. The two deterministic contiguous partitions have losses $7/48$ and $1/8$. For the randomized problem, let the two levels be $u \leq v$. Any choice with $v < 1/4$ can be improved by raising its levels. For $v \geq 1/4$, feasibility requires $u \leq 1/4$; raising u to $1/4$ weakly increases every relevant concave chord value. Thus take $u = 1/4$.

For v in $[1/4, 1/2]$, $[1/2, 3/4]$, and $[3/4, 1]$, respectively, Theorem 5.1 gives loss

$$\begin{cases} [(1/2 - v)(2 - v) + (3/4 - v)(2 - v)]/3, \\ [(v - 1/2)/8 + (3/4 - v)(2 - v)]/3, \\ [(v - 1/2)/4 + (v - 3/4)/2]/6. \end{cases}$$

The first two expressions strictly decrease on their domains and the third strictly increases. Hence the global minimum is at $v = 3/4$ and equals $1/96$. The first and third branches use their respective endpoint symbols; the middle branch uses each with probability one half. All intermediate tiers are zero. The middle branch's loss is the strict-concavity loss $1/32$, weighted by its probability $1/3$. Three distinct terminal symbols attain full information. $\square$

The institutional qualification is substantive. On the middle branch, the high private draw delivers terminal payment $3/4$ against an expected-continuation cap $1/2$. The lottery is feasible when participation is checked before the private draw, as stipulated here, but not when that branch cap must hold for every private

draw. We do not transfer the randomized frontier to the pathwise institution. The deterministic frontier and the exact-memory lower bound retain their original domains. This separation explains why a statement about the minimum alphabet at the exact optimum cannot settle the economic value of randomization under a smaller alphabet.

6 An Exact Global Randomized Memory Frontier

The fixed-codebook formula in Theorem 5.1 leaves a continuous design problem: both the terminal service levels and their assignment probabilities must be chosen. We now solve that problem globally for quadratic terminal rewards and heterogeneous quadratic intermediate costs. The solution also determines when imposing participation after, rather than before, the private draw eliminates the value of randomization. These are restricted-alphabet design results, distinct from the exact-implementation alphabet in Theorem EC.4.1.

Retain the three-date renewal architecture of Theorem 4.1, including the saturated root promise $\bar{b} = \sum_{j=1}^k \pi_j b_j$, strictly ordered caps $0 < b_1 < \dots < b_k < 1$, and positive probabilities summing to one. Throughout this section,

$$f(c) = rc - \frac{q}{2}c^2, \quad h_j(y) = \frac{\gamma_j}{2}y^2, \quad r \geq q > 0, \quad \gamma_j \geq 0. \quad (7)$$

The parameters and probabilities can differ across branches as indicated. Terminal reward is strictly increasing on the feasible cap range, including when $r = q$. All private information retained until the common terminal vertex must be encoded in the charged symbol. The decoder and public observation convention are unchanged.

6.1 Cap anchors and one continuous terminal level

For $u \leq b \leq v$, quadratic concavity gives the interpolation loss

$$f(b) - \frac{v-b}{v-u}f(u) - \frac{b-u}{v-u}f(v) = \frac{q}{2}(b-u)(v-b), \quad (8)$$

with the endpoint value understood when levels coincide. Thus an interior codeword enters the loss affinely until it crosses a branch cap. The highest codeword is different: reducing it below a branch cap creates an intermediate payment and its associated cost.

Lemma 6.1 (Cap-anchored representation). *For every alphabet budget $m \geq 1$, an optimum of (5) exists with the following representation. Its lowest codeword is b_1 ; every codeword except the highest belongs to $\{b_1, \dots, b_k\}$; and the highest belongs to $[b_1, b_k]$. A one-symbol optimum is $\{b_1\}$. The highest codeword need not be a branch cap.*

Proof. Attainment follows from the compact level-and-probability formulation in Theorem 5.1. Delete unused levels and coincident duplicates. If all levels lie below b_1 , raising the largest to b_1 improves every branch and leaves only that level relevant. Otherwise delete levels below b_1 except the largest such level, if any, and raise it to b_1 . Every affected cap is bracketed by this level and the next one. Equation (8) shows that raising the lower endpoint weakly lowers loss. This also covers a one-level codebook. The same argument

at the other extreme deletes superfluous levels above b_k and lowers the remaining highest endpoint to b_k without increasing loss.

Fix the highest level. Take a nonhighest codeword not equal to a cap. Between the nearest caps and its neighboring codewords, the set of branch caps in each adjacent interpolation interval is fixed. Equation (8) makes the sum of their losses affine in the moving codeword; all other losses are unchanged. Move it to an endpoint that does not increase this affine function. At a cap it becomes anchored. At a collision it can be deleted: no strictly intervening branch cap has been crossed, and the limiting interpolation losses agree. Each move reduces the number of unanchored nonhighest levels or the number of levels. Finitely many moves produce the stated representation. Branches above the fixed highest level are unaffected throughout. $\square$

The lemma does not discretize the highest level. A cap-only restriction can be strictly suboptimal, as Proposition 6.3 shows. It also does not assert that every optimal codebook is anchored; it supplies an optimum with this useful form.

6.2 A finite exact algorithm

For adjacent anchored levels $b_i < b_j$, define

$$E(i, j) = \frac{q}{2} \sum_{h=i}^j \pi_h (b_h - b_i)(b_j - b_h). \quad (9)$$

Let $P(s, j)$ be the minimum interpolation loss on caps through b_j using exactly s anchored levels, starting at b_1 and ending at b_j . Set $P(1, 1) = 0$, $P(1, j) = +\infty$ for $j > 1$, and use the ordered recurrence

$$P(s, j) = \min_{i < j} \{P(s-1, i) + E(i, j)\}, \quad 2 \leq s \leq j. \quad (10)$$

This is a standard shortest-path or segmentation recurrence. The substantive reduction is Lemma 6.1, which makes it applicable to a jointly optimized continuous lottery codebook.

Suppose that b_i is the last anchored level and z is the final level. For $\ell \geq i$ and $b_\ell \leq z \leq b_{\ell+1}$, put $b_{k+1} = b_k$ and define

$$\begin{aligned} \Psi_{i\ell}(z) &= \frac{q}{2} \sum_{h=i+1}^{\ell} \pi_h (b_h - b_i)(z - b_h) \\ &\quad + \sum_{h=\ell+1}^k \pi_h \left\{ f(b_h) - f(z) + \frac{\gamma_h}{2} (b_h - z)^2 \right\}. \end{aligned} \quad (11)$$

The first sum is the lottery loss below z ; the second is the terminal-reward and intermediate-payment loss above it. Empty sums are zero. For $\ell < k$ the function is strictly convex in z and has the constrained minimizer

$$z_{i\ell} = \left[\frac{r \sum_{h>\ell} \pi_h + \sum_{h>\ell} \pi_h \gamma_h b_h - \frac{q}{2} \sum_{h=i+1}^{\ell} \pi_h (b_h - b_i)}{q \sum_{h>\ell} \pi_h + \sum_{h>\ell} \pi_h \gamma_h} \right]_{[b_\ell, b_{\ell+1}]}. \quad (12)$$

For $\ell = k$, take $z_{ik} = b_k$. Define the terminal cost

$$T_i = \min_{i \leq \ell \leq k} \Psi_{i\ell}(z_{i\ell}). \quad (13)$$

Theorem 6.2 (Global frontier and exact complexity). *Under (7), the optimum with at most m symbols is*

$$D_m^{\text{ran}} = \min \left\{ D_1, \min_{\substack{1 \leq s \leq \min(m-1, k) \\ 1 \leq j \leq k}} \{P(s, j) + T_j\} \right\}, \quad (14)$$

where the inner minimum is empty for $m = 1$. Backtracking produces a globally optimal codebook and the adjacent lotteries implementing it. For ordered inputs and $1 \leq m \leq k$, all frontiers through budget m can be computed in $O(mk^2)$ exact rational operations and $O(mk + k)$ stored rational numbers and indices, including backpointers. No codeword grid is required.

For rational input of total binary length L , a reduced-rational implementation has polynomial Turing complexity. A conservative bound is $O(mk^2S^3)$ bit operations with $S = O(k[L + \log(k + 2)])$. This includes exact cost comparisons, clamping, and fraction reduction. For $m \geq k$ the optimal loss is zero; listing additional budget entries requires only their output cost. These guarantees concern the stated algorithm, not an arbitrary nonlinear optimizer for (5).

Proof. By Lemma 6.1, a nontrivial optimal codebook consists of an anchored prefix and one final level. The prefix contributes exactly the sum of its edge costs. Splitting at the preceding anchor proves (10) by induction on the number of anchors. If two successive edges share an endpoint cap, its loss is zero, so no term is double counted. Conditional on the final anchor b_j , every possible highest level belongs to one of the intervals in (11). Differentiating that quadratic gives (12); its quadratic coefficient is strictly positive whenever $\ell < k$ because the remaining probabilities and q are positive. Formula (13) therefore optimizes over the entire remaining continuous domain.

Every term in (14) reconstructs a feasible controller with at most m codewords. A coincident final level is deleted. Conversely, the representation lemma maps an optimal controller into one of those terms, including the one-symbol alternative. The two inequalities prove equality, not merely an upper bound or a stationary-point condition. Recover the final level attaining T_j , backtrack the minimizing indices in P , and apply Theorem 5.1 to obtain the encoder probabilities and intermediate tiers.

Prefix sums of π_h , $\pi_h b_h$, $\pi_h b_h^2$, $\pi_h \gamma_h$, $\pi_h \gamma_h b_h$, and $\pi_h \gamma_h b_h^2$ evaluate each edge and each quadratic tail in constant arithmetic work. All T_i require $O(k^2)$ work. The prefix dynamic program requires $O(mk^2)$ work and $O(mk)$ backpointers; edge costs are evaluated on demand rather than stored in a quadratic array. The six moment arrays and all terminal optima require $O(k)$ storage.

For completeness, each moment, edge coefficient, and tail coefficient is a sum of bounded-degree monomials in rational inputs. A conceptual common denominator is a fixed power of the product of input denominators; it has polynomial bit length and is not constructed by the implementation. Each terminal minimizer adds one division of such coefficients. Each prefix cost adds at most k edges; each final candidate adds one tail optimum. Reducing after every operation gives the conservative height bound S . Exact comparisons use cross multiplication on integers of $O(S)$ bits, and ordinary integer arithmetic

with greatest-common-divisor reduction costs at most $O(S^3)$ per rational operation. This establishes the stated bound, including comparisons rather than only arithmetic counts. The electronic companion gives an implementation-level concordance. $\square$

6.3 A genuinely continuous optimal codebook

Proposition 6.3 (An off-cap terminal level is optimal). *Let $(b_1, b_2, b_3) = (1/4, 1/2, 3/4)$, $(\pi_1, \pi_2, \pi_3) = (7/20, 3/5, 1/20)$, $r = 2$, $q = 1$, and $\gamma_j = 1$. The globally optimal two-symbol expected-participation codebook is $(1/4, 5/8)$, and*

$$D_2^{\text{ran}} = \frac{23}{1280} < \frac{3}{160} = \min_{\substack{|c| \leq 2 \\ c \subseteq \{b_1, b_2, b_3\}}} D(c) = D_2, \quad (15)$$

where the cap-only minimum is over feasible codebooks and $D(c)$ denotes their expected-participation loss. Thus even optimizing all cap-only lotteries misses the unrestricted optimum.

Proof. The representation lemma fixes the lower level at $1/4$. For an upper level $z \in [1/2, 3/4]$, the loss is

$$\frac{3}{40}(z - 1/2) + \frac{1}{20}(3/4 - z)(2 - z) = \frac{z^2}{20} - \frac{z}{16} + \frac{3}{80}.$$

Its unique minimum is at $z = 5/8$ and equals $23/1280$. On $[1/4, 1/2]$, both higher-cap branches lie above z and the loss strictly decreases up to $1/2$. Levels above $3/4$ can be lowered, as in Lemma 6.1. These observations also exclude the one-level boundary. The cap-only choices with two feasible levels have losses $3/160$ and $3/160$; one level has loss $49/160$. The deterministic partition using levels $1/4, 1/2$ has loss $3/160$, while the other contiguous two-cell partition has loss $21/80$. Theorem 4.1 yields the deterministic optimum.

The middle branch draws terminal level $5/8$ with probability $2/3$ and level $1/4$ with probability $1/3$, with zero intermediate tier. The highest-cap branch uses intermediate tier $1/8$ and terminal tier $5/8$. All branch payments equal their caps in expectation, and the root payment is $17/40$. The resulting weighted losses are $3/320$ and $11/1280$, respectively, summing to $23/1280$. $\square$

6.4 Participation after the private draw

The preceding frontier is appropriate when a renewal review approves an expected continuation before the protocol draws its private symbol. Consider instead an agreement that requires the same cap to hold for every private realization after that review. A draw-specific intermediate tier is allowed; only the terminal public state and the charged symbol reach the terminal decoder. Write $D_m^{\text{pw,ran}}$ for the best randomized loss under these pathwise constraints and the same saturated root promise.

Theorem 6.4 (Pathwise randomization has no frontier advantage). *For the general renewal family of Theorem 4.1, with strictly increasing concave f and convex nondecreasing h_j , and the observation architecture just specified,*

$$D_m^{\text{pw,ran}} = D_m \quad \text{for every } m \geq 1. \quad (16)$$

Consequently (14) and the deterministic frontier give an exact comparison of the two participation institutions under (7). Their exact-optimum alphabet thresholds are both k when strict concavity makes the k distinct full-information actions unique.

Proof. Pathwise feasibility gives $Y_j + C_j \leq b_j$ almost surely at each branch. The root promise is $\sum_j \pi_j b_j$ and every branch has positive probability. Hence $Y_j + C_j = b_j$ almost surely on every branch: any positive expected slack would contradict root saturation.

A retained symbol determines a terminal level, after replacing an independent randomized terminal decoder by its mean. This replacement preserves feasibility and weakly improves reward. To see this under pathwise saturation, conditional on a branch and symbol the only permissible additional terminal randomization is fresh and independent of the earlier tier; otherwise a retained shared seed would be an uncharged state. Independence together with $Y_j + C_j = b_j$ almost surely forces that decoder to be degenerate. This also shows why the memory architecture matters.

For a fixed codebook, a symbol with terminal level c can be used on branch j only if $c \leq b_j$; saturation fixes its intermediate tier at $b_j - c$. Its payoff is $f(c) - h_j(b_j - c)$, which is strictly increasing in c . Thus the highest eligible codeword weakly dominates every lottery over eligible symbols, including lotteries with draw-specific intermediate tiers. Selecting it deterministically preserves each branch's payment and cannot reduce payoff. Optimizing codebooks yields the deterministic ordered frontier of Theorem 4.1. The reverse inequality follows because every deterministic feasible controller is an admissible pathwise-randomized controller. Strict-concavity uniqueness gives the final threshold assertion. $\square$

This institutional result is not an assertion that randomization is generally ineffective in dynamic contracts. It uses the saturated root promise, an exogenous recombination, and the charged-information interface. Under expected participation, Theorem 6.2 chooses a less expensive lottery protocol; under pathwise participation, the same service model calls for the deterministic frontier. In either institution an operator can price a symbol budget, add its implementation charge to the computed loss, and optimize the budget without confusing read-only program size with retained private information.

7 Monge Costs and Classical Linear Matrix Search

The finite dynamic program still optimizes a continuous highest service level. We first prove the contractual cost inequalities and then verify the classical easy-direction completion with the exact order used by the code. A search routine cannot establish the economic reduction that supplies its entries.

7.1 The inequality survives continuous terminal minimization

Write $I_\ell = [b_\ell, b_{\ell+1}]$, including $I_k = \{b_k\}$, and $Q(i, \ell) = \min_{z \in I_\ell} \Psi_{i\ell}(z)$ for $i \leq \ell$. A partial array is Monge here when

$$A(r, c) + A(s, d) \leq A(r, d) + A(s, c), \quad r < s, \quad c < d, \quad (17)$$

whenever all four entries are defined. This fixes the inequality convention before any orientation is named.

Lemma 7.1 (Contractual finite-cost inequalities). *The anchored edge costs E and deterministic cell costs C satisfy $A(a, c) + A(b, d) \leq A(a, d) + A(b, c)$ for $a \leq b \leq c \leq d$. The continuously minimized terminal array Q is Monge on its feasible domain. These statements permit arbitrary positive probabilities and heterogeneous nonnegative intermediate curvatures.*

Proof. For E , expansion of (9) gives the right-minus-left difference

$$\frac{q}{2} \left[(b_d - b_c) \sum_{h=a}^{b-1} \pi_h (b_h - b_a) + (b_b - b_a)(b_d - b_c) \sum_{h=b}^c \pi_h + (b_b - b_a) \sum_{h=c+1}^d \pi_h (b_d - b_h) \right] \geq 0.$$

Every term is nonnegative, including coincident-index and empty-sum cases. For C , all terms through c cancel and the difference is

$$\sum_{h=c+1}^d \pi_h \{f(b_b) - f(b_a) + h_h(b_h - b_a) - h_h(b_h - b_b)\} \geq 0.$$

Only monotonicity of f and h_h is needed for the second inequality.

Join the functions $\Psi_{i\ell}$ to a continuous function Ψ_i on $[b_i, b_k]$. For $i < j$ and $z \geq b_j$, cancellation gives

$$\begin{aligned} \Psi_i(z) - \Psi_j(z) = \frac{q}{2} & \left[\sum_{h=i+1}^j \pi_h (b_h - b_i)(z - b_h) \right. \\ & \left. + (b_j - b_i) \sum_{b_j < b_h \leq z} \pi_h (z - b_h) \right]. \end{aligned} \quad (18)$$

The expression is continuous and nondecreasing in z : between caps its slope is nonnegative, and every entering summand is zero at its entry cap. Intermediate-curvature terms cancel. Thus $\Psi_i(x) + \Psi_j(y) \leq \Psi_i(y) + \Psi_j(x)$ for $b_j \leq x \leq y$.

For feasible columns $\ell < t$ with $j \leq \ell$, let x minimize Ψ_j on I_ℓ and y minimize Ψ_i on I_t . Compactness ensures attainment and interval order gives $x \leq y$, including a common endpoint. Therefore

$$Q(i, \ell) + Q(j, t) \leq \Psi_i(x) + \Psi_j(y) \leq \Psi_i(y) + \Psi_j(x) = Q(i, t) + Q(j, \ell).$$

The singleton final interval is evaluated directly. This proves the inequality after continuous optimization; convexity of each separate minimization would not by itself suffice. $\square$

Lemma 7.2 (Previous-layer values are column offsets). *Adding any finite column term v_c preserves the Monge inequality. For fixed finite predecessor values, the arrays $A(j, i) = P(s, i) + E(i, j)$ and $B(j, i) = D(s - 1, i) + C(i + 1, j)$ are therefore Monge wherever all four entries are feasible.*

Proof. The two column terms occur once on each side of (17) and cancel. For rows $j < j'$ and columns $i < i'$ with $i' < j$, apply Lemma 7.1 to $i < i' < j < j'$ for E , and $i + 1 < i' + 1 \leq j < j'$ for C . Transposition

of the cost arguments gives the required inequality. Unreachable predecessor values are removed, not treated as finite offsets. $\square$

7.2 Completion with the implemented strict and tie orders

Give a feasible entry the lexicographic key

$$K(r, c) = (0, A(r, c), c). \quad (19)$$

Finite ties select the smaller column, and distinct columns have distinct keys. The exact total-monotonicity condition used by the implementation is

$$K(r, d) < K(r, c) \implies K(s, d) < K(s, c), \quad r < s, c < d. \quad (20)$$

A strict preference for the right column cannot reverse in a later row. This applies to arbitrary selected rows and columns. Because keys are distinct, a decrease of row-argmin indices in any submatrix would contradict (20).

Lemma 7.3 (Suffix-feasible completion). *Let row r have feasible columns $c \geq \ell_r$, with nondecreasing cutoffs ℓ_r . Suppose the finite entries are Monge. Complete every $c < \ell_r$ with $K(r, c) = (1, 0, -c)$. Then (20) holds. A completely infeasible row of a selected submatrix selects its largest selected column.*

Proof. Fix $r < s$ and $c < d$ satisfying the antecedent. There are three possible upper-row configurations.

If both entries are feasible, the strict preference means $A(r, d) < A(r, c)$; equality would select c . If both remain feasible in row s , Monge gives $A(s, d) - A(s, c) \leq A(r, d) - A(r, c) < 0$. If c becomes infeasible but d remains feasible, d wins by its zero first key coordinate. If both become infeasible, $-d < -c$ makes d win. A feasible c with infeasible d is impossible for a suffix.

If c is infeasible and d feasible in row r , the nondecreasing cutoff keeps c infeasible in row s . Either d remains feasible and wins by its first coordinate, or both are padded and $-d < -c$ wins. Finally, if both are infeasible in row r , both remain infeasible in row s and the same padding order applies. These cases are exhaustive. In an entirely infeasible selected row, all keys are $(1, 0, -c)$, uniquely minimized at the largest selected column. Thus rows becoming entirely infeasible after column restriction are included, not excluded by an implicit feasibility assumption. $\square$

Lemma 7.4 (Prefix-feasible completion). *Let row r have feasible columns $c \leq u_r$, with nondecreasing cutoffs u_r . Suppose the finite entries are Monge. Complete every $c > u_r$ with $K(r, c) = (1, 0, c)$. Then (20) holds. A completely infeasible selected row selects its smallest selected column.*

Proof. If $d > u_r$, either c is feasible and wins by its first key coordinate, or both are padded and $c < d$ wins. The antecedent is false in either case. If the antecedent is true, both upper-row entries are finite. Since $u_s \geq u_r$, both remain finite in row s . A finite tie cannot favor d in row r , and the strict finite inequality propagates by Monge exactly as above. On a wholly padded selected row, $(1, 0, c)$ is uniquely minimized at

its smallest column. This covers finite ties, feasible/padded comparisons, and padded/padded comparisons without assigning a numerical infinity. $\square$

These lemmas specialize the known favorable completion setting in Section 2; they are not general partial-matrix search theorems. A helper accepting two bounds does not prove total monotonicity for arbitrary interval rows. For example, two zero-cost rows feasible only at columns 2 and 1 have a vacuously Monge finite part but decreasing minima. Uniform infinite padding with ordinary left ties also fails on the required suffix domain: rows with cutoffs 2, 3 restricted to columns 1, 2 select 2, 1. The opposite internal padding orders above are deliberate.

7.3 Reachability and three search interfaces

Lemma 7.5 (Contiguous reachable predecessor sets). *For the anchored program, $P(1, j)$ is finite exactly at $j = 1$, and for $s \geq 2$, $P(s, j)$ is finite exactly when $s \leq j \leq k$. For the deterministic program, $D(0, j)$ is finite only at $j = 0$, and $D(s, j)$ is finite exactly when $s \leq j \leq k$ for $s \geq 1$. Each globally reachable predecessor set that can extend to a further endpoint is a contiguous interval or the initial singleton.*

Proof. An anchored path with s distinct indices ending at j requires $j \geq s$. Conversely, $1, 2, \dots, s-1, j$ gives a finite path whenever $j \geq s \geq 2$. The one-anchor condition fixes $j = 1$. A deterministic partition of j branches into s nonempty ordered cells exists exactly when $j \geq s$. These are necessary and sufficient characterizations. Discarding endpoint k , which cannot extend to a later endpoint, preserves interval structure. $\square$

The following pseudocode uses one-based cap indices and zero for the empty deterministic prefix. “Search” means ordinary full-matrix SMAWK on the supplied key. All prior-layer values are known before a search.

Terminal interface. Rows and columns are $1, \dots, k$. At (i, ℓ) return $(1, 0, -\ell)$ if $\ell < i$; otherwise minimize (11) by (12) and return $(0, Q(i, \ell), \ell)$. Search all rows. Reevaluate each winning cell once to store T_i and its continuous optimizer.

Anchored-prefix interface. To form layer $s+1$, use rows $j = s+1, \dots, k$. Columns are $\{1\}$ for $s = 1$, and $s, \dots, k-1$ otherwise. At (j, i) return $(1, 0, i)$ if $i \geq j$, and $(0, P(s, i) + E(i, j), i)$ otherwise. Search and store values and predecessors. Scan the current reachable endpoints with T_i to update the at-most-budget incumbent; backtrack only its winning codebook.

Deterministic-prefix interface. To form layer s , use rows $j = s, \dots, k$. Columns are $\{0\}$ for $s = 1$, and $s-1, \dots, k-1$ otherwise. Return $(1, 0, i)$ if $i \geq j$, and $(0, D(s-1, i) + C(i+1, j), i)$ otherwise. Search, store predecessors, and backtrack the solution ending at k .

Lemma 7.5 justifies every discarded column. The terminal interface has shrinking feasible suffixes; the program interfaces have expanding feasible prefixes. The completion lemmas include infeasible rows in submatrices created internally by SMAWK, although every original searched row has a feasible entry. Padded comparisons do not evaluate an undefined economic cost.

Theorem 7.6 (Exact complete frontiers with linear work per layer). *For ordered rational inputs satisfying (7) and $1 \leq m \leq k$, both expected-participation and deterministic/pathwise frontiers through budget m , including an optimal codebook for every budget, require $O(mk)$ rational operations and comparisons and $O(mk + k)$ stored rational numbers and indices. The highest randomized level remains continuous. With total input length L and $S = O(k[L + \log(k + 2)])$, a conservative Turing bound is $O(mkS^3)$. Sorting unordered distinct caps adds $O(k \log k)$ exact comparisons.*

Proof. The finite-cost, offset, completion, and reachability lemmas verify all hypotheses of classical SMAWK (Aggarwal et al., 1987). Each interface call uses $O(k)$ key comparisons. A request either compares padded indices or performs constantly many moment operations and exact cost evaluations, including repeated requests without caching. The six moment arrays cost $O(k)$ work and space. The terminal pass and at most m program layers cost $O(mk)$. Winner reevaluation and last-anchor scans add $O(mk)$ rather than an uncounted quadratic search.

There are $O(mk)$ predecessors, two active value rows, and $O(k)$ extra search indices. Backtracking one winner for each budget writes at most $\sum_{s=1}^m s = O(m^2) \subseteq O(mk)$ symbols. Reconstruction and output are thus included.

Search introduces no new algebraic depth. A terminal call forms the same six moments, coefficient ratio, clamped level, and substituted quadratic value as Theorem 6.2. A program call adds one edge or cell to a prior path, and a selected path has at most k such costs. Comparisons evaluate a subset, possibly with repetition, of the candidates of quadratic enumeration. Column offsets change no formula; padding uses indices only. The common-denominator argument of that theorem therefore still bounds evaluated rationals and lottery probabilities by $O(S)$ bits. Reduced-fraction arithmetic and cross multiplication cost conservatively $O(S^3)$ per operation. Table 2 records the full accounting. $\square$

The retained divide-and-conquer implementation is a valid $O(mk \log(k + 1))$ alternative using ordinary row-minimum monotonicity. It is an implementation comparator rather than another headline theorem. Linear rational work does not imply uniform wall-time superiority for a particular Python implementation.

8 Pricing Information and Implementation

8.1 A frontier is a resource-allocation primitive

The symbol budget is not a synonym for total memory. Let I specify a participation institution, m a declared alphabet capacity, and t a read-only representation from a stated implementation menu. For the fixed rational input, suppose the menu charges $R_{I,t}(m, S)$ read-only bits, $Q_{I,t}(m, S)$ evaluation work per use, and $C_{I,t}(m, S)$ offline work. Here S is the coefficient-height bound, and the same charges apply to every loss-minimizing controller represented in that menu entry. Public-state storage and any branch-record access are specified

separately. For nonnegative resource prices and $H > 0$ uses, the design objective is

$$\min_{I,m,t} \{D_m^I + \lambda_w \lceil \log_2 m \rceil + \lambda_R R_{I,t}(m, S) + \lambda_T Q_{I,t}(m, S) + C_{I,t}(m, S)/H + \kappa_I\}. \quad (21)$$

The quantity κ_I is a stipulated cost of implementing the participation protocol, not an estimated willingness to accept risk. Restrict the menu to feasible $m \leq k$ and implementations satisfying the interface in Section 3. The exact frontiers and finite enumeration solve (21). This follows by minimizing service loss within each menu entry and then comparing entries; no additional convexity claim is needed.

The qualification on the menu is important. When read-only cost depends on the particular codebook rather than the declared capacity and representation, the loss-minimizing codebook need not minimize loss plus storage. Then (21) is a restricted-menu decision, and a joint controller/representation search is needed for a larger claim. We do not identify a one-dimensional loss frontier with that unrestricted multidimensional optimum.

For example, storing m rational decoder levels requires $O(mS)$ bits, but a direct branch encoder can require $O(kS)$ more. A general compact-response implementation can trade explicit per-vertex tables for a shared post-compilation circuit, as proved in the companion; this changes both read-only storage and evaluation work. Neither a free public vertex nor a linear writable alphabet makes those quantities disappear. Offline compilation and the exact-rational frontier calculation are also distinct costs. Table 2 records what the frontier theorem includes rather than charge a general compiler to every renewal instance.

When writable bits are the only variable implementation cost, the best controller with w bits uses the frontier at capacity $\min(2^w, k)$. It need not physically use every available symbol. The three equal-probability branches in Proposition 5.2 give a transparent decision example:

$$D_1 = 7/16, \quad D_2^{\text{ran}} = 1/96, \quad D_2^{\text{pw}} = 1/8, \quad D_3^{\text{ran}} = D_3^{\text{pw}} = 0.$$

A one-bit controller can use two symbols; a two-bit controller can implement all three distinct terminal actions. With no other variable charges, every bit price strictly between $1/96$ and $1/8$ makes one bit optimal under expected participation and two bits optimal under pathwise participation. Comparing with the zero-bit loss $7/16$ confirms that the zero-bit option does not overturn either choice. The institutional distinction therefore changes an actual capacity decision, not only a reported objective value.

8.2 An intermediate realization requirement

An operator may permit a bounded payment overrun without accepting the full expected-participation lottery. Consider the explicit protocol in which the intermediate tier Y_j is committed before the ticket draw, and impose

$$\mathbb{E}[Y_j + C_j] = b_j, \quad Y_j + C_j \leq b_j + \delta \quad \text{almost surely}, \quad \delta \geq 0. \quad (22)$$

This is an additional timing restriction for the intermediate institution, not a claim that pre-draw intermediate service is without loss for every bounded-risk model. It is without loss in the original expected-participation

optimum. The pathwise endpoint agrees with the stronger theorem allowing draw-specific intermediate actions.

For a fixed codebook, let u be its largest level not exceeding b_j . If the next level $v > b_j$ exists and $v - b_j \leq \delta$, the optimal branch uses the adjacent lottery with mean b_j and takes $Y_j = 0$. Otherwise it takes $C_j = u$ and $Y_j = b_j - u$. A level equal to b_j is used deterministically. Proposition A.1 proves this rule; in particular, it does not replace an almost-sure bound with an expected one.

The off-cap example in Proposition 6.3 admits an exact global solution under this intermediate protocol. With at most two symbols, define

$$z_\delta = \frac{1}{2} + \min\{\delta, \frac{1}{8}\}, \quad D_2^\delta = \frac{z_\delta^2}{20} - \frac{z_\delta}{16} + \frac{3}{80}. \quad (23)$$

The optimal codebook is $(1/4, z_\delta)$. Thus the deterministic endpoint has loss $3/160$, whereas allowing an overrun of $1/8$ attains the expected-participation optimum $23/1280$. Between those endpoints, the optimal highest service level moves continuously with the permitted overrun. The proof in Proposition A.2 optimizes over all codebooks with at most two levels; the computational checks are not its justification. We do not extend the general cap-anchoring or linear-frontier theorem to arbitrary bounded-overrun instances.

At the unrestricted optimum the middle branch receives $5/8$ with probability $2/3$ and $1/4$ otherwise. Its possible overrun is exactly $1/8$, and its expectation remains $1/2$. The highest branch takes intermediate service $1/8$ and terminal service $5/8$ deterministically. This makes the risk allocation and the timing of service explicit. A hard cap admits the zero-overrun endpoint, not the same lottery described with different terminology.

9 Verification and Computational Decision Support

The computational study has three distinct targets: correctness of the implemented interfaces, practical costs of alternative implementations, and evaluation of a finite reduced model by an external solver. It does not calibrate demand or estimate customers' willingness to accept realization variation. The deployment mechanism in Section 3 and its prices are stipulated design inputs.

Exact checks. The new tests enumerate monotone cutoffs in both completion directions, including wholly padded selected rows, finite ties, signed column offsets, and every nonempty row and column submatrix through dimension four. They compare direct minima with both the retained implementation and the independently authored offline search in Eppstein (2005). Separate heterogeneous rational instances compare all budgets against the quadratic and divide-and-conquer engines and replay every returned controller directly. Bounded-overrun tests use an independent all-pairs lottery calculation and include nonanchored codebook grid checks. The proofs, rather than these finite tests, establish the continuous and all-size statements.

The executed suite checks 784 completed matrices and 282,832 selected-submatrix row minima, including 98,728 wholly padded selected rows. It records 1,890 exact frontier equalities and direct controller replays, 190 bounded-overrun all-pairs equalities, and 33 evaluations of the solved tolerance frontier. The

unchanged R33–R36 suites also pass in isolated directories. All 24 independently assembled HiGHS cases yield an exactly certified reduced-network optimum and a matching controller.

Scale and memory. The same-input study extends the inherited family to 16,384 branches. For branch index $j = 1, \dots, k$, it uses $b_j = j/(k + 1)$, $\pi_j = 2j/[k(k + 1)]$, $\gamma_j = (j \bmod 5)/3$, and $r = 2, q = 1$, computing budgets one through eight. Each engine and institution runs in a fresh process with an explicit 120-second process limit. A limit observation is reported as such, never as a measured frontier time. Table 3 reports unprofiled time, economic requests, and peak resident memory. The independent PADS comparison substitutes search alone; economic moments and reconstruction remain shared and are not mislabeled independent algorithms.

At the largest completed common size for expected participation, 16,384 branches, the retained SMAWK implementation takes 26.601 seconds against 37.436 for divide-and-conquer, a time ratio of 0.711. At the largest completed common size for pathwise participation, 16,384 branches, the retained SMAWK implementation takes 44.132 seconds against 52.054 for divide-and-conquer, a time ratio of 0.848. Across the 6 completed same-size comparisons, SMAWK is slower in 2. These are observations on one runner and one input family, not a uniform speedup claim. No configured study case exceeded the stated process limit.

Python reduced-fraction arithmetic makes exact equality and coefficient growth auditable, but it is not a performance model for an optimized native implementation. Independent heap-traced runs measure Python-tracked allocations; resident peaks also include the interpreter and imported code. Separately profiled runs decompose exclusive time into rational arithmetic, economic-oracle work, matrix-search/key bookkeeping, and the combined frontier-scan/reconstruction routine. The last category is deliberately not presented as a separately measured reconstruction cost. All current measurements and their definitions are reported in the article and its appendices.

External finite-model comparison. We also assemble unit-flow networks directly from branch sums, without the moment oracle, and solve them with compiled HiGHS dual simplex through SciPy (Huangfu and Hall, 2018; Virtanen et al., 2020). The expected-participation network uses the proved continuous terminal minimizations, not a discretization of the highest level. HiGHS operates in floating point. We extract its integral path, reprice it in exact fractions, verify the controller, and certify its reduced-network optimum by exact shortest-path potentials. Table 4 separates graph construction, solver time, and exact certification plus replay. This is an external solver comparison for the finite reduction, not a claim that the solver independently proves cap anchoring or that native SMAWK has been benchmarked.

The earlier experiments, including unfavorable small-instance search constants and shared-cache results, remain unchanged in the preservation archive. The new evidence addresses the remaining implementation questions rather than accumulate synthetic examples as a substitute for operational data. In particular, finite-precision wall time, rational-operation complexity, and contract value are reported as different quantities.

10 Conclusions

Limited retained information creates a service-design frontier, and participation timing changes that frontier. With approval before a private draw, the provider can improve a restricted codebook by using adjacent lotteries and optimizing its highest level continuously. With realization-by-realization caps, randomization cannot improve any budget in the stated renewal architecture. A bounded-overflow agreement connects these alternatives in an exactly solved example, while explicit bit prices show when the institution changes the optimal writable capacity.

The model-specific optimization step is the cap-anchored reduction and the preservation of Monge structure after continuous terminal minimization. Its favorable one-sided domains then admit classical matrix search, yielding complete exact frontiers with linear rational work per budget layer. The resulting values support a stated implementation menu that prices writable bits, read-only representation, evaluation work, and compilation separately. Those distinctions are essential: neither a small symbol alphabet nor a better asymptotic search bound alone establishes a cheaper deployed service system.

Code, data, and reproducibility. The accompanying repository contains one current code directory and one reproduction entry point, `code/reproduce.py`, within the R37 revision package. It supplies exact engine snapshots, the attributed PADS excerpt, direct-network HiGHS comparisons, tests, and machine-readable measurements. The root article and electronic companion are the current readers. The preservation map locates every inherited result and the exact preceding readers; old theorem modules and evidence are unchanged. A build manifest binds the current sources, measurements, reader hashes, page counts, and whole-tree preservation checks to a scientific source commit. All experiment inputs are synthetic and public. No journal submission or editorial acceptance is represented by the repository build.

A Proofs for the Bounded-Overflow Institution

The results in this section use the pre-draw intermediate-action protocol of (22). They do not assert that an arbitrary draw-dependent intermediate-action model reduces to this protocol. All terminal levels and intermediate actions are in $[0, 1]$.

Proposition A.1 (Exact fixed-codebook bounded-overflow rule). *Let f be increasing concave and h_j convex nondecreasing, as in Theorem 4.1. Fix a sorted codebook with lowest level at most b_1 . On branch j , let u be the largest codeword at most b_j . If $u = b_j$, use u deterministically. Otherwise, if there is a next codeword $v > b_j$ and $v - b_j \leq \delta$, an optimal choice is the adjacent-code lottery with mean b_j , with $Y_j = 0$. In every other case an optimal choice is the deterministic codeword u , with $Y_j = b_j - u$.*

Proof. Put $\mu = \mathbb{E}C_j$. The committed intermediate action is $Y_j = b_j - \mu \geq 0$, and every codeword used with positive probability must satisfy $c \leq \mu + \delta$. The greatest expected terminal payoff at a fixed mean is the concave polygonal interpolation $F_c(\mu)$ through the codebook. The function

$$G_j(\mu) = F_c(\mu) - h_j(b_j - \mu)$$

is nondecreasing, and strictly increasing when f is strictly increasing. If $u = b_j$, the deterministic action attains the maximal possible mean. If the top codeword is at most b_j , it similarly attains the maximal possible mean and is optimal.

For the remaining case, $u < b_j < v$. If $v - b_j \leq \delta$, the adjacent lottery with mean b_j satisfies the support bound: its high realization is at most $b_j + \delta$, and its low realization is smaller. It attains both the largest allowable mean and $F_c(b_j)$, and hence the upper bound $G_j(b_j)$.

If $v - b_j > \delta$, no feasible mean can exceed u . Such a mean would require positive mass at a codeword at least v , whereas the realization requirement would imply $v \leq \mu + \delta \leq b_j + \delta$, a contradiction. The deterministic action at u is feasible, attains $G_j(u)$, and weakly dominates every feasible smaller mean. The argument does not permit a tiny mass on an inadmissible high codeword: an almost-sure constraint excludes any positive mass there. $\square$

Proposition A.2 (Global two-code bounded-overflow example). *For the probabilities, caps, and quadratic rewards in Proposition 6.3, the minimum loss over all codebooks with at most two levels under (22) is (23). In particular, the optimal highest level is continuous in δ on $[0, 1/8]$ and is off-cap for every $\delta > 0$.*

Proof. We first justify restrictions on the two codewords; they are not imposed by the proposed answer. Feasibility requires the lowest codeword to be at most $b_1 = 1/4$. If both codewords are below $1/4$, increasing the highest to $1/4$ improves every branch's deterministic maximal mean without violating a realization bound. Otherwise write the two levels as $u \leq 1/4 \leq z$. Raising u to $1/4$ cannot worsen an optimal branch payoff. For a feasible mean $\mu \geq 1/4$, use the new adjacent mixture with the same mean: its high endpoint z and the overrun condition are unchanged, while the concave quadratic interpolation increases as the low endpoint rises. A feasible mean below $1/4$ is dominated by deterministic service $1/4$, which all branch caps permit. Thus an optimum has lowest codeword $1/4$, including a one-code solution as a repeated level.

A highest level above $b_3 = 3/4$ can be reduced to $3/4$. For any branch mean at most $3/4$, the adjacent interpolation improves as the upper endpoint falls, and the maximum realization decreases. Conversely, if $z < 1/2$, both the second and third branches are at the top codeword and improving z to $1/2$ strictly raises their objective with intermediate service adjusted to keep total payment equal to the cap. The first branch remains at $1/4$. Hence it suffices to optimize $z \in [1/2, 3/4]$.

If $z > 1/2 + \delta$, Proposition A.1 forces the middle branch to use the lower codeword deterministically. Its weighted loss alone is

$$\frac{3}{5}\{f(1/2) - f(1/4) + \frac{1}{2}(1/2 - 1/4)^2\} = \frac{21}{80}.$$

This exceeds the total feasible loss $3/160$ achieved by $(1/4, 1/2)$. Such a codebook is never optimal. For $z \in [1/2, \min\{3/4, 1/2 + \delta\}]$, the middle branch instead uses its mean-preserving adjacent lottery, while the highest branch takes deterministic terminal service z and intermediate service $3/4 - z$. Direct substitution gives

$$\Psi(z) = \frac{z^2}{20} - \frac{z}{16} + \frac{3}{80}, \quad \Psi'(z) = \frac{z}{10} - \frac{1}{16}.$$

The unique unconstrained minimum is $z = 5/8$. Clamping it to the proven feasible interval gives $z_\delta = 1/2 + \min\{\delta, 1/8\}$. The one-code loss is $49/160$, larger than the two-code value throughout. The displayed

formula is therefore the global at-most-two-code optimum. It reduces to $3/160$ at $\delta = 0$ and $23/1280$ for $\delta \geq 1/8$. $\square$

The last statement about being off-cap refers to the level itself, not a discrete number of feasible candidates. In particular $5/8$ remains off-cap for every larger δ as well. This example demonstrates a continuously moving operational design under a bounded realization policy. It is not a proof that the general bounded-overrun codebook problem has the same Monge structure as the expected-participation problem.

B Current Computational Methods and Their Scope

B.1 Exact search verification

The generic completion suite enumerates every nondecreasing cutoff sequence in dimensions one through four. Both shrinking feasible suffixes and expanding feasible prefixes are tested. Finite arrays have the form $r^2 + a_c - trc$, with $t \in \{0, 1\}$ and either zero or alternating signed column offsets. The zero-slope cases deliberately create ties. Every strict preference implication is checked, and both search implementations are compared with direct minima on every nonempty row and column restriction. Completely infeasible selected rows are retained and counted. Two negative controls reject uniform infinite suffix padding with left ties and arbitrary two-sided feasible intervals. These are finite tests of the proof’s cases, not substitutes for the all-size lemmas.

The economic suite combines the deterministic benchmark family with heterogeneous random rational instances generated from seed 370924. It compares all budgets against the quadratic engine and checks direct branch payoffs, saturation, codebook size, and randomized controller replay. The bounded-overrun reference enumerates all pairs of levels, rather than assume the adjacent-code conclusion. A separate grid permits nonanchored lower codewords in the solved risk example. Continuous optimality rests on the analytic propositions.

B.2 Independent matrix-search substitution

The offline ConcaveMinima function of Eppstein (2005) returns column minima. Transposing its interface produces the row-minimum oracle required here. The executable function is excerpted without algorithmic changes from the pinned PADS source; its MIT notice and original blob identity are in `pads_excerpt.py`. The experiment replaces only matrix search. Moment construction, contractual cost evaluation, and reconstruction remain the same. This isolates implementation dependence in the search layer and is not presented as an independent proof of the economic model. The inherited R33–R36 suites run unchanged in temporary directories, so their historical result files are never overwritten.

B.3 External finite-network solver

For deterministic participation, a network node represents a layer and an endpoint; an arc appends a nonempty contiguous cell at its direct branchwise loss. For expected participation, a node represents a count and last

cap-anchored codeword; an arc appends another anchor at its chord loss. Sink arcs add the continuously minimized terminal cost. A direct source-to-sink arc represents the one-code solution, and the network permits all budgets up to the requested bound. Every cost is assembled from direct rational sums rather than the moment oracle. Continuous terminal minimization is performed interval by interval with its analytically clamped quadratic minimizer.

The node-arc incidence matrix and unit source/sink demand give a linear program. Compiled HiGHS dual simplex is called through SciPy. The floating solution must be within 10^{-7} of an integral path. The selected path is then checked exactly: its fraction-valued cost equals the sink distance from an independent rational shortest-path pass, and every reached arc satisfies the potential inequality. Direct controller replay and comparison with the exact frontier complete the audit. Thus the recorded reduced-network objective has an exact certificate even though the external optimizer uses floating point. The certificate does not re-prove that this finite network represents the continuous contract problem.

B.4 Runtime and memory accounting

Each timed case runs in a new process and computes budgets one through eight. Input construction has a separate timer. Frontier time includes moment setup, every search and reevaluation, scans, and all winning-codebook reconstructions. It excludes direct validation and serialization, which occur after the measurements. The 120-second limit applies to the entire worker process; an exceeded limit is not a completed solver time. Hardware and interpreter versions are recorded with source hashes.

Peak resident memory is Linux process high-water memory in kibibytes, captured before validation; it includes interpreter, input, arrays, and outputs. Separate tracing begins before input construction and records Python-tracked current and peak bytes. It does not measure all native allocations. Traced runtimes are not compared with untraced times. Table 5 reports the traced cases.

Separate cProfile runs at 512 branches assign exclusive time to rational arithmetic (including gcd), moment/economic-oracle work, matrix-search and key bookkeeping, and the combined outer frontier scan/reconstruction routine. Inclusive function times are retained in JSON but not added across categories. The outer category cannot distinguish scanning from reconstruction and is labeled accordingly. Table 6 reports these exclusive measurements. Single-run timings describe this implementation and runner; they are not a statistical claim of hardware-independent superiority.

References

- Aggarwal A, Klawe MM, Moran S, Shor P, Wilber R (1987) Geometric applications of a matrix-searching algorithm. *Algorithmica* 2:195–208. doi:10.1007/BF01840359.
- Chan TM (2021) (Near-)linear-time randomized algorithms for row minima in Monge partial matrices and related problems. *Proceedings of the 2021 ACM–SIAM Symposium on Discrete Algorithms*, 1465–1482. doi:10.1137/1.9781611976465.88.
- Emmerich MTM (2026) Exact and fast subset selection algorithms for the bi-objective integral R2 indicator. Preprint, arXiv:2606.23365.
- Eppstein D (2005) PADS: Python Algorithms and Data Structures. *SMAWK.py*, offline matrix-search implementation, originally written 2002 and revised 2005. University of California, Irvine. MIT-licensed distribution; source version and excerpt are identified in the code supplement.
- Federgruen A, Groenevelt H (1986) The greedy procedure for resource allocation problems: Necessary and sufficient conditions for optimality. *Operations Research* 34(6):909–918. doi:10.1287/opre.34.6.909.
- Givan R, Dean T, Greig M (2003) Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence* 147(1–2):163–223. doi:10.1016/S0004-3702(02)00376-4.
- Groenevelt H (1991) Two algorithms for maximizing a separable concave function over a polymatroid feasible region. *European Journal of Operational Research* 54(2):227–236. doi:10.1016/0377-2217(91)90300-K.
- Huangfu Q, Hall JAJ (2018) Parallelizing the dual revised simplex method. *Mathematical Programming Computation* 10(1):119–142. doi:10.1007/s12532-017-0130-5.
- Kaplan H, Mozes S, Nussbaum Y, Sharir M (2017) Submatrix maximum queries in Monge matrices and partial Monge matrices, and their applications. *ACM Transactions on Algorithms* 13(2), Article 26. doi:10.1145/3039873.
- Klawe MM, Kleitman DJ (1990) An almost linear time algorithm for generalized matrix searching. *SIAM Journal on Discrete Mathematics* 3(1):81–97. doi:10.1137/0403009.
- Mjelde KM (1983) Resource allocation with tree constraints. *Operations Research* 31(5):881–890. doi:10.1287/opre.31.5.881.
- Peyrière J (2023) Moore machines duality. *Theoretical Computer Science* 951:113774. doi:10.1016/j.tcs.2023.113774.
- Spear SE, Srivastava S (1987) On repeated moral hazard with discounting. *Review of Economic Studies* 54(4):599–617. doi:10.2307/2297484.

- Tang CS (1990) Reducing separable convex programs with tree constraints. *Management Science* 36(11):1407–1412. doi:10.1287/mnsc.36.11.1407.
- Thomas J, Worrall T (1988) Self-enforcing wage contracts. *Review of Economic Studies* 55(4):541–554. doi:10.2307/2297404.
- Virtanen P, Gommers R, Oliphant TE, et al. (2020) SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272. doi:10.1038/s41592-019-0686-2.
- Yao FF (1980) Efficient dynamic programming using quadrangle inequalities. *Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing*, 429–435. doi:10.1145/800141.804691.
- Zhang H (2012) Solving an infinite horizon adverse selection model through finite policy graphs. *Operations Research* 60(4):850–864. doi:10.1287/opre.1120.1056.

Table 1: Contractual structure and inherited algorithmic machinery.

Result in this article	Contribution and precise comparison
Deterministic cells and pathwise participation	Theorem 4.1 gives the contractual cell cost and ordered sufficiency. Theorem 6.4 proves that even private pathwise randomization cannot improve it; the segmentation recurrence itself is standard.
Global continuous code-books	Lemma 6.1 and Theorem 6.2 establish cap anchoring except at the highest level and optimize that level exactly. These are not consequences of generic matrix search. Proposition 6.3 rules out cap-only discretization.
Terminal costs after minimization	Lemma 7.1 proves a contractual single-crossing identity with cancellation of heterogeneous intermediate curvatures. It implies Monge structure after continuous interval minimization, which does not follow merely from separate convexity.
Completed row-minimum search	Lemmas 7.3–7.4 audit the exact keys in the classical easy-direction staircase case described by Chan (2021). SMAWK is due to Aggarwal et al. (1987); generalized partial search and partial-query structures have their own literature (Klawe and Kleitman, 1990; Kaplan et al., 2017).
Complete linear-work frontier	Theorem 7.6 combines the contractual oracle and reachability with classical search. It is not a new general partial-Monge algorithm. Emmerich (2026) illustrates the same broad Monge-DP acceleration pattern in another application.
Broader exact representation	The companion preserves the compact-response, tight-size, piecewise-quadratic, and exact-memory theorems. Laminar allocation, memoization, and behavioral minimization are explicitly attributed rather than relabeled.

Table 2: Accounting for Theorem 7.6; ordered rational input, $1 \leq m \leq k$.

Cost component	Included work or storage
Input validation and six prefix moments	$O(k)$ rational operations; $O(k)$ stored values.
Sorting	Input interface requires ordered distinct caps; otherwise add $O(k \log k)$ exact comparisons and reorder associated primitives.
Continuous terminal minimization	Included in $O(k)$ oracle requests; each interval minimizes its quadratic exactly, including singleton and endpoint cases.
Key comparisons and repeated requests	Included in $O(mk)$ work; a repeated finite request is charged again. No quadratic entry cache is assumed.
Padding comparisons	Included index comparisons; undefined economic costs are never evaluated.
Previous-layer values and backpointers	$O(mk)$ stored values and indices, plus $O(k)$ working search space.
Reconstruction and output	One winning codebook per budget; $O(m^2) \leq O(mk)$ symbols written. No reconstruction at every candidate endpoint.
Rational heights and integer operations	$O(S)$ bits per evaluated rational, $S = O(k[L + \log(k+2)])$; conservative $O(mkS^3)$ bit operations, including reduction and comparison.
Physical bit storage	$O((mk + k)[S + \log(k+2)])$ bits for the stated arrays and indices; not a claim of $O(mk)$ bits.
Inherited public graph and response compiler	Not invoked by the renewal algorithm; their read-only data, compilation, and query costs enter the separate implementation menu when used.
External solver preprocessing	Not part of the frontier theorem. Finite-network construction, floating solver work, and exact certification are measured separately.

Table 3: Same-input unprofiled frontier measurements; budgets one through eight.

Institution	Branches	Search	Seconds	Requests	Resident MiB
Expected	256	Divide-conquer	0.3136	12,177	22.7
Expected	256	SMAWK	0.3452	15,376	22.6
Expected	256	PADS	0.3387	14,754	22.7
Pathwise	256	Divide-conquer	0.4071	13,524	22.8
Pathwise	256	SMAWK	0.5904	19,386	22.7
Pathwise	256	PADS	0.5525	18,127	22.6
Expected	4,096	Divide-conquer	7.9101	296,018	36.0
Expected	4,096	SMAWK	6.4134	259,034	33.2
Expected	4,096	PADS	6.3043	247,709	33.4
Pathwise	4,096	Divide-conquer	10.9931	335,422	36.3
Pathwise	4,096	SMAWK	10.7799	329,278	32.4
Pathwise	4,096	PADS	10.1865	307,277	32.8
Expected	16,384	Divide-conquer	37.4357	1,381,513	69.0
Expected	16,384	SMAWK	26.6007	1,039,837	69.1
Expected	16,384	PADS	26.1597	994,145	70.6
Pathwise	16,384	Divide-conquer	52.0540	1,572,352	65.4
Pathwise	16,384	SMAWK	44.1321	1,322,658	66.1
Pathwise	16,384	PADS	41.4011	1,234,124	67.4

Requests count finite economic-oracle evaluations, including reevaluations. Resident memory is the process peak before validation and includes its baseline. A limit denotes the 120-second whole-process limit, not a measured frontier time.

Table 4: External compiled solver on independently assembled reduced networks; eight-symbol budget.

Institution	Branches	Arcs	Construction	Solver	Certification
Expected	8	92	0.0041	0.0022	0.0055
Pathwise	8	101	0.0023	0.0022	0.0051
Expected	16	488	0.0300	0.0044	0.0165
Pathwise	16	585	0.0146	0.0047	0.0204
Expected	32	2,240	0.2074	0.0139	0.0453
Pathwise	32	2,897	0.1048	0.0164	0.0620
Expected	64	9,584	1.5880	0.0559	0.1337
Pathwise	64	12,897	0.7787	0.0675	0.1701

All times are seconds. Certification includes exact repricing, rational shortest-path potentials, controller replay, and comparison with the exact frontier. The machine-readable record also includes budgets two and four. Solver time excludes network construction and certification.

Table 5: Separate memory runs at 1,024 branches and budgets one through eight; mebibytes.

Institution	Search	Current heap	Peak heap	Resident peak
Expected	Divide-conquer	2.60	3.92	33.4
Expected	SMAWK	0.59	2.62	29.2
Expected	PADS	0.65	2.68	29.3
Pathwise	Divide-conquer	1.96	2.88	29.7
Pathwise	SMAWK	0.53	2.50	29.3
Pathwise	PADS	0.59	2.58	29.3

Tracing starts before input creation and stops before validation. Heap fields are Python-tracked allocations; resident peaks include the process baseline and native allocations. Traced runtimes are not used as the unprofiled comparison.

Table 6: Exclusive-time decomposition in separate 512-branch profile runs; percentages.

Institution	Search	Arithmetic	Oracle	Search	Outer	Other
Expected	Divide-conquer	69.5	11.8	1.6	0.2	16.9
Expected	SMAWK	67.5	10.6	5.2	0.1	16.5
Expected	PADS	67.0	10.6	4.6	0.2	17.7
Pathwise	Divide-conquer	72.0	3.6	8.0	0.1	16.3
Pathwise	SMAWK	70.5	3.5	9.8	0.0	16.2
Pathwise	PADS	70.0	3.5	9.5	0.0	17.0

Arithmetic includes reduced-fraction operations and gcd. Oracle and search columns count their own exclusive work, not child arithmetic again. Outer combines frontier scans and reconstruction. Percentages use exclusive profile time and are not percentages of unprofiled wall time; rounding can prevent an exact sum of 100.